VoiceCmd-DTW

Isolated-Word Speech Recognition using MFCC Features and Dynamic Time Warping

PROJECT SUBMISSION REPORT - SPEECH RECOGNITION

Project Overview

This project presents the design and implementation of **VoiceCmd-DTW**, a speaker-dependent, template-based isolated-word speech recognition system. Unlike deep-learning ASR services, every stage of the classical pipeline is implemented from first principles with NumPy and SciPy: MFCC feature extraction, delta features, cepstral mean/variance normalisation, energy-based end-point detection, and Dynamic Time Warping with a Sakoe-Chiba band. A Streamlit web application lets a user enrol a vocabulary from uploaded or recorded WAV audio, recognise new utterances, inspect the signal, features and DTW alignment, and evaluate accuracy with a confusion matrix.

Submission Details

Field	Information
Subject	Speech Recognition
Topic	Implementation of an Isolated-Word Speech Recognition System using MFCC and Dynamic Time Warping
Source Code	Project folder **voicecmd-dtw/**, submitted together with this report
Documentation	**README.md** in the project folder (installation, usage, CLI, architecture)
Demonstration	Run **streamlit run app.py**; one-click demo vocabulary and demo test clips are included

Student Details

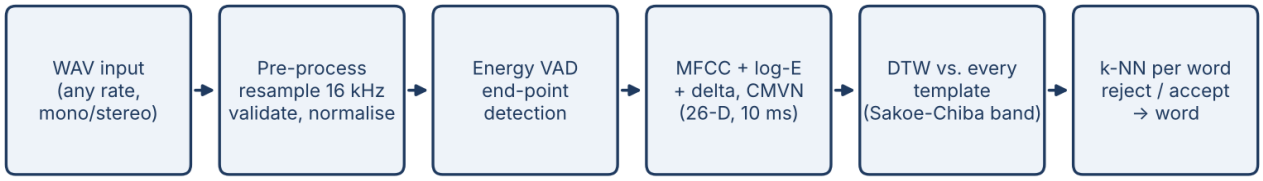
S. No.	Student Name	Registration Number
1	HIRTHIK P	RA2311003050442

Key Features and Deliverables

Component	Description
Audio Input and Validation	Loads 8/16/32-bit PCM and float WAV files (mono or stereo), resamples to 16 kHz, removes DC offset, normalises level, and rejects empty, corrupt, too-short, too-long or silent clips. Microphone recording is supported in the app.
Feature Extraction	From-scratch MFCC: 25 ms Hamming frames, 10 ms hop, 26-filter mel bank, 12 liftered cepstra + log energy, first-order deltas (26 dimensions) and per-utterance CMVN.
End-Point Detection	Adaptive energy-threshold voice activity detection that trims leading/trailing silence, ignores clicks and keeps closures inside a word.
DTW Recognition	Symmetric-step DTW with a Sakoe-Chiba band, row-vectorised via a cumulative-minimum identity; k-nearest-template scoring per word and an optional rejection threshold calibrated by leave-one-out for out-of-vocabulary words.
Results and Performance	Predicted word, DTW distance, confidence margin, waveform, MFCC heat-map and warping path; audio duration, processing time and real-time factor; JSON export, model save/load (.npz), accuracy and confusion matrix for evaluation sets.
Testing and Reproducibility	42 automated tests (pytest) cover audio I/O, features, DTW, VAD, recognition, persistence and a headless UI flow; evaluation is fully scripted and seeded (scripts/evaluate_synthetic.py).

Technology Stack: Python · NumPy · SciPy · Streamlit · Matplotlib · pytest

System Architecture



The recogniser stores one MFCC sequence per enrolled recording (a *template*). An unknown utterance is converted to the same representation and aligned to every template with DTW, which absorbs differences in speaking rate. The distance is the accumulated path cost normalised by the sum of both sequence lengths; the vocabulary word whose best template(s) give the smallest distance is returned.

Evaluation Protocol

Because no recorded corpus is bundled, the system is evaluated on **synthetic formant speech** produced by the included generator (6 command words - go, stop, left, right, up, down - rendered by pseudo-speakers that differ in pitch, vocal-tract length and speaking rate). Templates: 5 speakers x 2 clean repetitions (10 per word). Tests: **180 utterances from 10 unseen speakers**, so the evaluation is speaker-independent with respect to the template set. These figures validate the implementation and show relative effects of design choices; they are not a claim about accuracy on natural human speech.

Table 1 - Noise robustness (white noise added to test clips only)

Test condition (SNR)	Accuracy	Mean real-time factor
Clean	100.0%	0.032
30 dB	100.0%	0.030
20 dB	99.4%	0.027
15 dB	92.8%	0.028
10 dB	87.2%	0.028
5 dB	72.2%	0.025
0 dB	42.8%	0.020

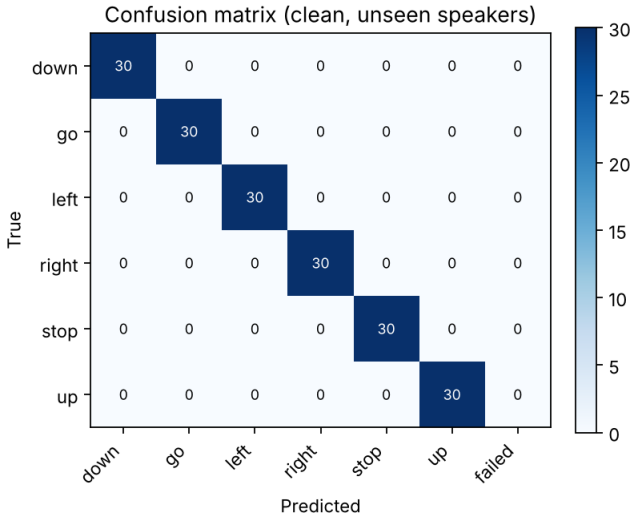
Table 2 - Ablation study (same templates, 180 test utterances per column)

Design variant	Clean	10 dB SNR
Baseline (MFCC+energy+delta, CMVN, VAD, band 0.3, k=1)	100.0%	87.2%
No delta features	100.0%	85.0%
No CMVN	99.4%	21.1%
No VAD (whole clip)	100.0%	96.7%
Narrow band (0.1)	100.0%	88.3%
No band (full DTW)	100.0%	86.7%
k = 3 nearest templates	100.0%	86.7%

Findings. CMVN is the single most important normalisation step (10 dB accuracy 21% without it versus 87% with it), and delta features give a modest gain (85.0% without them). The band constraint changes accuracy only marginally while keeping cost bounded. Under noise the energy-based VAD is the weakest component: disabling it raised 10 dB accuracy to 96.7% at the price of a higher real-time factor, which motivates a noise-robust VAD as future work. For out-of-vocabulary rejection, a threshold calibrated from the templates (4.34) accepted 100% of 50

in-vocabulary utterances and rejected 100% of 30 utterances of a word that was not enrolled.

Figure 1 - Confusion matrix (clean test set, unseen speakers)



Scope and Limitations

VoiceCmd-DTW is a small-vocabulary, template-matching recogniser: it recognises one isolated word per clip from the words that were enrolled, and its accuracy on real speech depends on enrolling several clear recordings per word from the same user. It is not a continuous-speech or open-vocabulary transcription system. Real recordings can be enrolled and evaluated with the app or the command-line tool without code changes.

Note: Reported results were produced by running scripts/evaluate_synthetic.py and the pytest suite on the submitted code; they can be regenerated with the commands in README.md.